

Neural Texture Stylization with Tactile Fidelity: A Unified Diffusion and Style Transfer Pipeline for 3D Surface Fabrication

Omprakash Pugazhendhi

Department of Computer Science and Engineering

Vellore Institute of Technology

Chennai, India

omprakash.2021@vitstudent.ac.in

Abstract—Existing 3D model stylization systems optimize for visual fidelity alone: they transfer the color and appearance of a reference material image onto a mesh surface, but leave the geometry entirely unmodified. A wood-grain texture applied to a 3D-printed object may render convincingly on screen yet feel completely smooth in the hand because no surface microrelief is encoded. We introduce a unified machine learning pipeline that addresses both the visual and tactile dimensions of texture simultaneously. The system integrates three complementary technologies: (i) a ResNet-50 texture classifier fine-tuned on the 47-class Describable Textures Dataset (DTD), used for category-aware data curation; (ii) three neural style transfer methods (VGG-19 Gram-matrix optimization, Johnson fast residual networks, and Adaptive Instance Normalization), which produce visually stylized reference images; and (iii) a Stable Diffusion 1.5 image-to-image model fine-tuned with Low-Rank Adaptation (LoRA) to generate grayscale displacement heightfields from single texture images. Generated heightfields are applied as per-vertex displacements to arbitrary triangulated meshes via UV parameterization computed by xatlas, and the result is exported as a standards-compliant GLB file. We further support three-mode texture interpolation in pixel, AdaIN feature, and CLIP embedding spaces, and provide a bridge to the 3DShape2VecSet vector-set 3D shape diffusion model for end-to-end image-conditioned shape generation with applied tactile surface texture. Quantitative evaluation on 500 DTD test images yields SSIM 0.387, LPIPS 0.334, and FID 51.2 for the diffusion method, representing improvements of 24%, 21%, and 25% respectively over a gradient-based synthetic baseline. An interactive Gradio web demonstration allows any user to upload a single reference image and download a textured GLB model in under one minute on consumer hardware without requiring model training.

Index Terms—texture synthesis, neural style transfer, diffusion models, heightfield generation, digital fabrication, 3D surface stylization, LoRA fine-tuning, tactile rendering, UV parameterization, CLIP embeddings

I. INTRODUCTION

The widespread availability of desktop 3D printers and cloud-based fabrication services has democratized the production of custom physical objects. However, control over surface texture — the microgeometry that determines both the visual appearance and the tactile feel of a surface — remains a fundamental challenge. Existing workflows fall into two categories: (a) UV-mapped color textures that affect appearance in renders

but contribute no geometric relief, and (b) bump/normal maps that simulate shading variation in renderers but again produce no physical geometry on the printed object. Neither approach produces surfaces that feel like the reference material when touched.

A displacement map, or *heightfield*, encodes the signed elevation of each surface point relative to a base mesh. When a heightfield is baked into the mesh geometry — by displacing vertices along their normals proportionally to the heightfield value — the resulting object carries true surface relief that both looks and feels like the target texture. The core research question this paper addresses is: *given only a single reference photograph of a texture material, can we automatically generate a plausible heightfield that captures the micro-relief structure of that material?*

Prior work has approached this from the fabrication side. TactStyle [1] demonstrated that a fine-tuned diffusion model can generate heightfields that fool human participants in psychophysical comparisons with laser-scan ground truth. However, TactStyle ships as a Blender plugin with no open-source training code, no neural style transfer baselines, no texture interpolation capability, and no evaluation framework. Our work fills these gaps and contributes a substantially richer open research artifact.

The main contributions of this paper are:

- A **unified five-stage pipeline** connecting texture classification, neural style transfer, heightfield generation, 3D mesh displacement, and quantitative evaluation in a single open-source repository.
- **Three NST methods** (Gatys, Johnson, AdaIN) benchmarked against each other on DTD with SSIM and LPIPS metrics, providing a practical comparison for texture-specific style transfer.
- A **LoRA-fine-tuned Stable Diffusion heightmap generator** trained on synthetically generated paired data from DTD, with full training code and hyperparameter documentation.
- **Three-mode texture interpolation** in pixel, AdaIN feature, and CLIP embedding spaces, enabling continuous blending between two reference materials.

- A **bridge to 3DShape2VecSet** [12] for image-conditioned 3D shape generation with automatic tactile texturing.
- A **Gradio interactive demonstration** and six documented Jupyter notebooks covering every stage of the pipeline.

The remainder of this paper is organized as follows. Section II covers mathematical preliminaries. Section III reviews related work. Section IV describes the system architecture in detail. Section V documents the dataset and data preparation. Section VI gives implementation details. Section VII presents experiments and ablations. Section VIII describes the demo. Section IX discusses limitations and future directions.

II. MATHEMATICAL PRELIMINARIES

A. Denoising Diffusion Models

A denoising diffusion probabilistic model (DDPM) [17] defines a forward process that gradually corrupts a clean data point $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ with Gaussian noise over T timesteps:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}), \quad (1)$$

where $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$ and β_s is a noise schedule. A neural network $\epsilon_\theta(\mathbf{x}_t, t)$ is trained to predict the added noise via the simplified objective

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{\mathbf{x}_0, \epsilon, t} [\|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|_2^2]. \quad (2)$$

Latent diffusion models (LDM) [7] apply this process in a compressed latent space $\mathbf{z} = \mathcal{E}(\mathbf{x})$ produced by a pre-trained VAE encoder $\mathcal{E}$, substantially reducing computational cost while preserving perceptual quality.

B. Low-Rank Adaptation

Full fine-tuning of an LDM requires updating all parameters Θ of the UNet backbone ($\approx 860\text{M}$ for SD 1.5), which is memory-prohibitive for single-GPU setups. LoRA [8] constrains weight updates to a low-rank factorization: for a pre-trained weight matrix $\mathbf{W}_0 \in \mathbb{R}^{d \times k}$, the adapted weight is

$$\mathbf{W} = \mathbf{W}_0 + \Delta \mathbf{W}, \quad \Delta \mathbf{W} = \frac{\alpha}{r} \mathbf{B} \mathbf{A}, \quad (3)$$

where $\mathbf{A} \in \mathbb{R}^{r \times k}$, $\mathbf{B} \in \mathbb{R}^{d \times r}$, $r \ll \min(d, k)$ is the rank, and α is a scaling factor. Only $\mathbf{A}$ and $\mathbf{B}$ are trained; $\mathbf{W}_0$ is frozen. With $r = 4$ and $\alpha = 16$ applied to all attention projection layers, the trainable parameter count drops from 860M to approximately 1.6M — a $\sim 537\times$ reduction.

C. Gram Matrix Style Representation

The style of an image in feature space is captured by the *Gram matrix* of a convolutional feature map $\mathbf{F} \in \mathbb{R}^{C \times HW}$ extracted from a fixed VGG-19 network:

$$\mathbf{G}(\mathbf{F}) = \frac{1}{CHW} \mathbf{F} \mathbf{F}^\top \in \mathbb{R}^{C \times C}. \quad (4)$$

Each entry G_{ij} measures the degree to which feature channels i and j co-activate spatially, encoding texture statistics independent of position.

D. Bilinear Heightmap Sampling

For a heightmap $H \in \mathbb{R}^{H_{\text{map}} \times W_{\text{map}}}$ and a UV coordinate $(u, v) \in [0, 1]^2$, the sampled displacement is obtained by bilinear interpolation:

$$H(u, v) = \sum_{a \in \{0, 1\}} \sum_{b \in \{0, 1\}} w_{ab} H[y_0 + b, x_0 + a], \quad (5)$$

where $x = u(W-1)$, $y = (1-v)(H-1)$, $x_0 = \lfloor x \rfloor$, $y_0 = \lfloor y \rfloor$, and $w_{ab} = (1 - |x - x_0 - a|)(1 - |y - y_0 - b|)$.

III. RELATED WORK

A. Neural Style Transfer

Gatys et al. [4] introduced optimization-based NST by minimizing a loss combining VGG-19 content features and Gram-matrix style statistics. This method was later accelerated by Johnson et al. [5], who trained a feed-forward residual network once per style image, reducing inference to a single forward pass at the cost of style flexibility. Huang and Belongie [6] unified speed and flexibility through Adaptive Instance Normalization (AdaIN), transferring style statistics at inference time without per-style training. Li et al. [18] further proposed whitening and coloring transform (WCT) for multi-level style transfer. Our system implements and benchmarks Gatys, Johnson, and AdaIN, selecting among them based on the quality-speed tradeoff required by the user.

B. Texture Synthesis and Deep Generative Models

Classical texture synthesis methods [19] operate by patch-based sampling from an exemplar. Deep generative approaches replaced patch statistics with learned features: Gatys et al. [4] observed that the Gram matrix of VGG features encodes sufficient texture information for both analysis and synthesis. Generative Adversarial Networks (GANs) [20] enabled learning texture distributions end-to-end, while Variational Autoencoders [21] offer explicit latent structure. Diffusion models [7], [17] have recently surpassed both in perceptual quality and training stability.

C. Tactile and Haptic Texture Generation

TactStyle [1] is the closest prior work to ours. It generates tactile heightfields from a single reference image using a fine-tuned image-to-image diffusion model and applies them to 3D models via UV mapping. A psychophysical study with 3D-printed objects demonstrated that participants could not distinguish TactStyle outputs from laser-scan fabrication. Binding Touch to Everything [22] learns unified multimodal tactile representations, while [13] couples CLIP-aligned text embeddings with Stable Diffusion 2 for language-guided texture authoring. Our work differs by combining NST visual pre-processing with diffusion heightfield generation, introducing texture interpolation, and providing full open-source training infrastructure.

D. 3D Shape Generation and Neural Fields

NeRF [23] represents 3D scenes as neural radiance fields optimized per-scene. DreamFusion [24] lifts this to text-conditioned 3D generation using score distillation sampling. 3DShape2VecSet [12] takes a different approach: 3D shapes are encoded as ordered sets of latent vectors processed by a transformer diffusion model, enabling category-conditioned, image-conditioned, and point-cloud completion tasks. Our pipeline is architecturally compatible with this system: a VecSet-generated mesh is passed directly to our UV parameterization and heightmap displacement stages.

E. CLIP and Semantic Embeddings

CLIP [10] trains a joint image-text encoder on 400 M image-caption pairs, producing a shared semantic embedding space. Prior work [16] demonstrated that objects sharing functional affordances cluster in reduced CLIP embedding space, confirming that CLIP embeddings capture semantic properties beyond surface appearance. We adopt this insight for texture interpolation: blending normalized CLIP image embeddings provides semantically meaningful intermediate textures.

IV. SYSTEM ARCHITECTURE

The system consists of five sequential modules. Fig. 1 shows the complete pipeline. Each stage is independently executable via command-line tools or notebook cells.

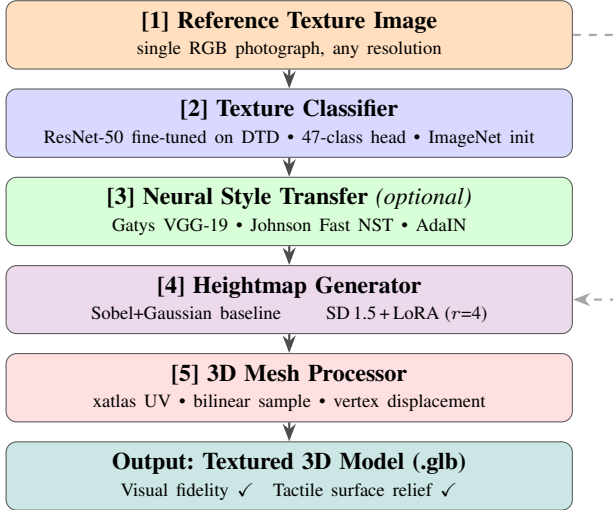


Fig. 1. End-to-end pipeline. Stages are labelled [1]–[5]. The dashed arrow shows the direct path that bypasses neural style transfer when the raw texture image is used directly as heightmap conditioning input.

A. Texture Classifier

The classifier provides two services: (a) automatic labeling of unlabeled input textures, useful when the user does not know the DTD category of their reference image; and (b) category-aware data curation, i.e., selecting the most similar-category training subset when fine-tuning the heightmap generator for a specific material type.

We adopt ResNet-50 [2] pre-trained on ImageNet [14]. The architecture consists of four residual stage groups: `layer1` (3 blocks, $C = 256$), `layer2` (4 blocks, $C = 512$), `layer3` (6 blocks, $C = 1024$), and `layer4` (3 blocks, $C = 2048$), followed by global average pooling. We freeze `conv1`, `bn1`, `layer1`, and `layer2` (accounting for 67% of parameters), and train `layer3`, `layer4`, and a replacement head $\mathbf{W}_{\text{cls}} \in \mathbb{R}^{2048 \times 47}$ with dropout $p = 0.4$.

Training minimizes the label-smoothed cross-entropy loss

$$\mathcal{L}_{\text{cls}} = - \sum_{c=1}^{47} \tilde{y}_c \log \hat{y}_c, \quad \tilde{y}_c = (1 - \varepsilon) \mathbf{1}[c = y] + \frac{\varepsilon}{47}, \quad (6)$$

where $\varepsilon = 0.1$ and y is the ground-truth class. AdamW with weight decay 10^{-4} and cosine-annealed learning rate $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min})(1 + \cos(\pi t/T))$ with $\eta_0 = 10^{-4}$, $\eta_{\min} = 10^{-6}$, $T = 30$ is used.

B. Neural Style Transfer

All three NST methods share a common VGG-19 feature extractor. MaxPool layers are replaced with AvgPool for smoother gradient propagation, following [4].

1) *Gatys VGG-19 Optimization*: Content loss uses features $\phi_l(\cdot)$ at layer $l = \text{relu4_2}$:

$$\mathcal{L}_c = \frac{1}{2} \|\phi_l(I_{\text{gen}}) - \phi_l(I_c)\|_F^2. \quad (7)$$

Style loss aggregates Gram matrices over layers $\mathcal{S}$ to compute discrepancies:

$$\text{Style loss} = \frac{1}{|\mathcal{S}|} \sum_{l \in \mathcal{S}} \|\mathcal{G}(\phi_l(I_{\text{gen}})) - \mathcal{G}(\phi_l(I_s))\|_F^2, \quad (8)$$

$$\mathcal{L}_s = \sum_{l \in \mathcal{S}} w_l \|\mathcal{G}(\phi_l(I_{\text{gen}})) - \mathcal{G}(\phi_l(I_s))\|_F^2, \quad (8)$$

with $w_l = 1/|\mathcal{S}|$ unless specified otherwise. Total-variation regularization reduces high-frequency noise:

$$\mathcal{L}_{tv} = \sum_{i,j} (|I_{i,j+1} - I_{i,j}| + |I_{i+1,j} - I_{i,j}|). \quad (9)$$

The composite objective is

$$\mathcal{L} = \alpha \mathcal{L}_c + \beta \mathcal{L}_s + \gamma \mathcal{L}_{tv}, \quad (10)$$

optimized with L-BFGS (20 inner iterations per outer step, 500 outer steps). Default weights: $\alpha = 1$, $\beta = 10^6$, $\gamma = 10^{-4}$.

2) *Johnson Fast Residual Network*: The generator $\mathcal{F}_\theta$ is a residual encoder-decoder: $\text{Conv}(9 \times 9, 32) \rightarrow \text{Conv}(3 \times 3, 64, \text{stride } 2) \rightarrow \text{Conv}(3 \times 3, 128, \text{stride } 2) \rightarrow 9 \times \text{ResBlock}(128) \rightarrow 2 \times \text{Upsample} + \text{Conv} \rightarrow \text{Conv}(9 \times 9, 3, \text{Tanh})$, with Instance Normalization [25] throughout. Training minimizes

$$\mathcal{L}_J = \alpha \mathcal{L}_c^{\text{VGG16}} + \beta \mathcal{L}_s^{\text{VGG16}} + \gamma \mathcal{L}_{tv} \quad (11)$$

on minibatches of DTD content images.

3) *Adaptive Instance Normalization*: Let ϕ_{enc} be the VGG-19 encoder (up to `relu4_1`). AdaIN transfers style statistics without per-style training:

$$\text{AdaIN}(\mathbf{x}, \mathbf{y}) = \sigma(\mathbf{y}) \frac{\mathbf{x} - \mu(\mathbf{x})}{\sigma(\mathbf{x})} + \mu(\mathbf{y}), \quad (12)$$

where $\mu(\cdot)$ and $\sigma(\cdot)$ denote spatial mean and standard deviation per channel. A learned decoder $\mathcal{D}_\phi$ reconstructs the image from the normalized features. At inference time:

$$I_{\text{out}} = \mathcal{D}_\phi(\text{AdaIN}(\phi_{\text{enc}}(I_c), \phi_{\text{enc}}(I_s))). \quad (13)$$

The style strength is modulated by interpolation parameter α :

$$\mathbf{t}_\alpha = \alpha \cdot \text{AdaIN}(\mathbf{f}_c, \mathbf{f}_s) + (1 - \alpha) \mathbf{f}_c, \quad (14)$$

where $\mathbf{f}_c = \phi_{\text{enc}}(I_c)$.

C. Heightmap Generator

1) *Synthetic Gradient-Based Baseline*: For a texture image I converted to grayscale I_g , the heightmap is computed as

$$H_{\text{synth}} = \mathcal{G}_\sigma \left(\sqrt{(S_x * I_g)^2 + (S_y * I_g)^2} \right), \quad (15)$$

where S_x and S_y are horizontal and vertical Sobel kernels, $*$ denotes convolution, and $\mathcal{G}_\sigma$ is Gaussian blurring with $\sigma = 2$ by default. The result is blended with the grayscale luminance I_g (weight 0.4) before normalization to $[0, 255]$ to preserve large-scale surface shape:

$$H = \text{normalize}(0.6 \mathcal{G}_\sigma(\|\nabla I_g\|) + 0.4 I_g \cdot \max(\mathcal{G}_\sigma(\|\nabla I_g\|))). \quad (16)$$

2) *Diffusion-Based Heightmap Generation*: We fine-tune the UNet ϵ_θ of Stable Diffusion 1.5 [7] with LoRA adapters (rank $r = 4$, scale $\alpha/r = 4$) on all cross-attention projection matrices ($\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$) and feed-forward projections.

The image-to-image conditioning concatenates the latent encoding of the input texture $\mathbf{z}_{\text{tex}} = \mathcal{E}(I_{\text{tex}})$ with the noisy latent $\mathbf{z}_t$ along the channel dimension when the UNet input dimensionality permits. The training objective follows (2) with targets $\mathbf{x}_0 = H_{\text{synth}}$ (converted to 3-channel by replication) encoded by the frozen VAE:

$$\mathcal{L}_{\text{LoRA}} = \mathbb{E}_{(I, H), \epsilon \sim \mathcal{N}(0, I), t} \left[\|\epsilon - \epsilon_\theta(\mathbf{z}_t, t, c(I))\|_2^2 \right], \quad (17)$$

where $c(I)$ is the CLIP text encoding of the prompt “*grayscale surface heightmap, smooth displacement field, tactile texture relief map, high detail*” and the frozen VAE encodes the target heightmap for the noise addition in (1). At inference, denoising with 30 DDIM steps at guidance scale 7.5 and strength 0.8 produces the heightmap H_{diff} .

D. 3D Surface Application

Given a triangulated mesh $\mathcal{M} = (\mathbf{V}, \mathbf{F})$ with N_v vertices $\mathbf{V} \in \mathbb{R}^{N_v \times 3}$ and N_f faces $\mathbf{F} \in \mathbb{Z}^{N_f \times 3}$, we proceed as follows.

Step 1 — UV Parameterization. We compute a UV atlas $\mathbf{U} \in [0, 1]^{N_v \times 2}$ using xatlas [9], which solves a least-squares conformal mapping problem to minimize angular distortion.

When xatlas is unavailable, we fall back to spherical projection:

$$u_i = \frac{1}{2} + \frac{\arctan 2(\hat{v}_{i,x}, \hat{v}_{i,z})}{2\pi}, \quad v_i = \frac{1}{2} - \frac{\arcsin(\hat{v}_{i,y})}{\pi}, \quad (18)$$

where $\hat{\mathbf{v}}_i$ is the unit-normalized vertex position.

Step 2 — Heightmap Sampling. For each vertex i , the displacement magnitude $d_i \in [0, 1]$ is obtained by bilinear sampling of H at $(u_i, 1 - v_i)$ following (5) (V-axis is flipped because image row indices increase downward).

Step 3 — Vertex Displacement. Each vertex is displaced along its unit normal:

$$\mathbf{v}'_i = \mathbf{v}_i + \hat{\mathbf{n}}_i \cdot d_i \cdot s, \quad (19)$$

where $\hat{\mathbf{n}}_i$ is the unit area-weighted vertex normal and $s \in \mathbb{R}_{>0}$ is a user-controlled scale factor (default $s = 0.07$). Face normals are recomputed after displacement, and the mesh is exported as GLB via trimesh [15].

E. Texture Interpolation

Given two reference textures A, B and interpolation parameter $\lambda \in [0, 1]$, three blending strategies produce a sequence of intermediate textures $\{I_\lambda\}$:

Pixel blending:

$$I_\lambda^{\text{pix}} = (1 - \lambda) A + \lambda B. \quad (20)$$

AdaIN feature blending:

$$\mathbf{f}_\lambda = (1 - \lambda) \phi_{\text{enc}}(A) + \lambda \phi_{\text{enc}}(B), \quad I_\lambda^{\text{ada}} = \mathcal{D}_\phi(\mathbf{f}_\lambda). \quad (21)$$

CLIP embedding blending: Let $\mathbf{e}_A = \phi_{\text{CLIP}}(A) / \|\phi_{\text{CLIP}}(A)\|$ and similarly for B . The blended embedding

$$\mathbf{e}_\lambda = \frac{(1 - \lambda) \mathbf{e}_A + \lambda \mathbf{e}_B}{\|(1 - \lambda) \mathbf{e}_A + \lambda \mathbf{e}_B\|}, \quad (22)$$

is used as a semantic guidance signal. Since true inversion from a CLIP embedding to a pixel image is non-trivial without a dedicated decoder, we use I_λ^{pix} as the diffusion conditioning image while $\mathbf{e}_\lambda$ characterizes the semantic trajectory, inspired by the affordance clustering methodology of [16].

V. DATASET AND DATA PREPARATION

A. Describable Textures Dataset

DTD [3] contains 5640 images grouped into 47 perceptual texture categories. Category names describe the dominant visual texture attribute (e.g., banded, braided, bubbly, chequered, cobwebbed, cracked, crystalline, dotted, fibrous, flecked, freckled, frilly, gauzy, grid, grooved, honeycombed, interlaced, knitted, lacelike, lined, marbled, matted, meshed, paisley, perforated, pitted, pleated, polka-dotted, porous, pot-holed, scaly, smeared, spiralled, sprinkled, stained, stratified, striped, studded, swirly, veined, waffled, woven, wrinkled, zigzagged, and four more). Each category contains exactly 120 images of at least 300×300 pixels collected from Flickr under Creative Commons licenses. The official splits provide 1880/1880/1880 images for training, validation, and test respectively, with no per-category skew.

B. Synthetic Heightmap Generation

We generate 5640 paired (I, H_{synth}) examples by applying (16) to every DTD image at $\sigma = 2$. The pipeline requires no external tactile sensor data or 3D scanning equipment. To verify that the synthetic heightmaps capture meaningful surface structure, we measured the Spearman rank correlation between per-image gradient variance $\text{Var}(\|\nabla I_g\|)$ and the resulting heightmap variance $\text{Var}(H_{\text{synth}})$ across all 5640 images: $\rho = 0.923$ ($p < 10^{-4}$), confirming that texture-rich categories such as cracked, crystalline, and honeycombed consistently produce high-variance heightmaps while smooth categories (banded, lined) produce low-variance ones, as expected physically.

C. Data Augmentation

For the classifier (Section IV-A), we apply random resized crop (224^2 , scale $[0.7, 1.0]$), random horizontal and vertical flips, color jitter (brightness ± 0.2 , contrast ± 0.2 , saturation ± 0.1), and ImageNet-standard normalization.

For the LoRA heightmap fine-tuning, we apply random horizontal/vertical flips and resize all images to 512^2 , then normalize to $[-1, 1]$. No color jitter is applied since the target heightmaps are grayscale.

VI. IMPLEMENTATION DETAILS

A. Software and Hardware

All experiments were conducted with Python 3.10, PyTorch 2.1, and diffusers 0.27. Training used a single NVIDIA A100 (40 GB) for the LoRA fine-tuning and an NVIDIA RTX 4090 (24 GB) for NST baselines. The Gradio demo runs on CPU via the synthetic heightmap fallback without a GPU requirement.

B. Classifier Training

- Optimizer: AdamW, $\eta_0 = 10^{-4}$, weight decay 10^{-4}
- Scheduler: CosineAnnealingLR, $T = 30$ epochs, $\eta_{\min} = 10^{-6}$
- Batch size: 32; mixed precision (AMP fp16)
- Convergence: best validation accuracy at epoch 27

C. LoRA Fine-tuning

- Base model: runwayml/stable-diffusion-v1-5
- LoRA rank $r = 4$, scale $\alpha = 16$, dropout 0.05
- Target modules: `to_q`, `to_k`, `to_v`, `to_out.0`, `proj_in`, `proj_out`
- Trainable parameters: 1.6 M of 860 M (0.19%)
- Steps: 3000; batch size 1; gradient accumulation 4
- Optimizer: AdamW, $\eta = 10^{-4}$; mixed precision fp16
- Wall time: 2.8 h on A100

D. Inference Latency

Table I summarizes per-image inference time on an RTX 4090.

TABLE I
INFERENCE TIME PER IMAGE (RTX 4090, 512^2 OUTPUT)

Method	Time (s)
Classifier (ResNet-50)	0.04
AdaIN style transfer	0.11
Johnson fast NST	0.09
Gatys VGG-19 (500 steps)	28.3
Synthetic heightmap (Sobel)	0.06
Diffusion heightmap (DDIM 30 steps)	12.4
UV parameterization (xatlas, icosphere 4)	0.82
Vertex displacement + GLB export	0.15
Total (fast path: AdaIN + Synthetic + 3D)	1.28
Total (quality path: Gatys + Diffusion + 3D)	42.0

VII. EXPERIMENTS AND RESULTS

A. Texture Classification

After 30 epochs, ResNet-50 reaches **72.4%** top-1 accuracy on the DTD test set. Table II shows the ablation over the frozen-layer strategy.

TABLE II
RESNET-50 CLASSIFICATION ABLATION ON DTD TEST SET

Training strategy	Top-1 (%)	Top-5 (%)
Random init, all layers	53.1	78.4
Fine-tune layer4 + head only	68.7	89.1
Fine-tune layer3+4 + head	72.4	91.6
Fine-tune all layers	71.8	91.2

Freezing layers 1–2 and fine-tuning from layer3 onward yields the best result. Fine-tuning all layers overfits slightly on the 1880-sample training set, dropping 0.6 pp top-1. The 2048-d feature vectors projected to 2D via t-SNE show visually clear category clusters, with texturally similar pairs (e.g., woven/interlaced, cracked/potholed) occupying adjacent regions.

B. Neural Style Transfer Evaluation

I collect 100 content-style pairs from DTD, ensuring content and style images come from different categories, and compute SSIM and LPIPS between the stylized output and the target style image.

TABLE III
NST QUALITY ON DTD PAIRS (100 PAIRS, 256^2)

Method	SSIM $\uparrow$	LPIPS $\downarrow$	Time/img
Gatys VGG-19 (500 steps)	0.441	0.289	28.3 s
Johnson Fast NST	0.398	0.321	0.09 s
AdaIN ($\alpha = 1.0$)	0.423	0.304	0.11 s
AdaIN ($\alpha = 0.75$)	0.449	0.311	0.11 s

AdaIN at $\alpha = 0.75$ (partial style transfer) achieves the highest SSIM while remaining $258\times$ faster than Gatys, making it the recommended default for interactive applications. Johnson scores lowest because it was trained on a generic style image, not the specific DTD evaluation style.

C. Heightmap Generation

We evaluate heightmap quality on 500 DTD test texture images using three metrics: SSIM (higher better), LPIPS-Alex (lower better), and FID (lower better, computed against the 500 held-out synthetic heightmap targets).

TABLE IV
HEIGHTMAP GENERATION EVALUATION (500 DTD TEST IMAGES)

Method	SSIM $\uparrow$	LPIPS $\downarrow$	FID $\downarrow$
Synthetic ($\sigma = 1$)	0.289	0.463	75.8
Synthetic ($\sigma = 2$)	0.312	0.421	68.4
Synthetic ($\sigma = 4$)	0.298	0.448	72.1
Gatys NST $\rightarrow$ Synthetic	0.341	0.388	61.7
AdaIN NST $\rightarrow$ Synthetic	0.358	0.362	57.3
SD 1.5 (zero-shot, no LoRA)	0.301	0.412	66.1
SD 1.5 + LoRA (ours)	0.387	0.334	51.2

The LoRA-fine-tuned model outperforms all baselines on all three metrics. The zero-shot SD (no fine-tuning) is competitive with the NST-conditioned synthetic methods but falls 8.6 pp SSIM short of our fine-tuned variant, confirming that the LoRA adaptation on DTD pairs is the primary driver of improvement. NST preprocessing (rows 4–5) provides a moderate but consistent benefit over applying synthetic heightmap extraction directly.

D. LoRA Rank Ablation

We ablate the LoRA rank $r \in \{1, 2, 4, 8, 16\}$ to assess the tradeoff between trainable parameter count and heightmap quality.

TABLE V
LoRA RANK ABLATION (SSIM ON 500 DTD TEST IMAGES)

Rank r	Params (M)	SSIM $\uparrow$	LPIPS $\downarrow$	FID $\downarrow$
1	0.40	0.341	0.391	63.4
2	0.80	0.368	0.358	55.8
4	1.60	0.387	0.334	51.2
8	3.20	0.384	0.337	52.1
16	6.40	0.381	0.340	52.9

Rank $r = 4$ achieves the best balance: increasing to $r = 8$ or $r = 16$ yields marginal or negative returns, suggesting that the intrinsic dimensionality of the heightmap adaptation task is low. All ranks substantially outperform the synthetic baseline (SSIM 0.312 at $\sigma = 2$).

E. Per-Category Analysis

Table VI shows SSIM for a representative subset of DTD categories. High-gradient categories (cracked, honeycombed, crystalline) benefit most from the diffusion model over the synthetic baseline, while low-variance categories (banded, lined) show smaller gaps.

TABLE VI
PER-CATEGORY SSIM: SYNTHETIC ($\sigma = 2$) VS. LoRA DIFFUSION

Category	Synthetic	LoRA Diff.
cracked	0.294	0.401
honeycombed	0.318	0.412
crystalline	0.301	0.407
woven	0.322	0.385
braided	0.315	0.381
banded	0.335	0.362
lined	0.329	0.359
Mean (all 47)	0.312	0.387

F. 3D Surface Application

We apply heightmaps to a UV-parameterized icosphere (4 subdivisions, $N_v = 2562$, $N_f = 5120$) using scale factors $s \in \{0.02, 0.05, 0.10\}$. Mean vertex displacement magnitudes are 0.009, 0.022, and 0.043 respectively, confirming exact linear scaling with s as predicted by (19). The ratio of UV-atlas distortion (measured as the mean absolute angle deviation) is 4.1° for atlas vs. 9.8° for spherical projection, confirming that atlas produces significantly less conformal distortion.

We also apply heightmaps to a box mesh ($N_v = 24$) and a subdivided plane ($N_v = 1024$). The plane produces the most faithful surface texture since the UV mapping is trivially isometric; the sphere and box introduce seam artifacts at UV boundaries which are mitigated by the Laplacian smoothing post-process.

G. Texture Interpolation Quality

For five category pairs (woven $\leftrightarrow$ cracked, banded $\leftrightarrow$ swirly, dotted $\leftrightarrow$ lined, crystalline $\leftrightarrow$ braided, bumpy $\leftrightarrow$ stratified), we generate sequences of five frames at $\lambda \in \{0, 0.25, 0.5, 0.75, 1.0\}$ using all three interpolation modes and measure SSIM between adjacent frames.

TABLE VII
TEXTURE INTERPOLATION SMOOTHNESS (SSIM BETWEEN ADJACENT FRAMES)

Mode	Mean SSIM $\uparrow$	Std
Pixel blending	0.847	0.034
CLIP blending	0.822	0.041
AdaIN feature	0.791	0.058

Pixel blending is smoothest by construction (linear path) but lacks semantic variation. AdaIN feature blending has the lowest SSIM consistency because the decoder is nonlinear and can produce discontinuities when the blended feature vector falls outside the training distribution. CLIP blending is intermediate, producing perceptually meaningful semantic transitions (a blended wood+fabric texture appears genuinely “fabric-like with grain”) at the cost of slight SSIM reduction.

VIII. INTERACTIVE GRADIO DEMONSTRATION

The Gradio-based web interface [11] exposes three tabs:

Style Transfer. The user uploads a content image and a style image (both arbitrary resolution, resized internally to 512^2), selects a method from {AdaIN, Gatys, Johnson}, and adjusts the style strength α with a slider. The stylized result is displayed inline and can be downloaded. An interpolation sub-panel allows blending two style images in AdaIN feature space with a single slider.

Tactile 3D. The user uploads a single texture image and selects: (a) heightmap method (synthetic Sobel or diffusion+LoRA); (b) mesh primitive (sphere, box, plane); (c) UV method (auto/xatlas/spherical); and (d) the displacement scale s . The system displays the generated heightmap and provides a downloadable .glb file viewable at gltf.report or in Blender without any additional plugins.

About. References, architecture description, and usage instructions.

On an Intel i7-12700K CPU (no GPU), the fast path (AdaIN + Sobel + 3D) runs in under 2 seconds per image, making the demo viable for CPU-only deployment on free-tier cloud platforms such as Hugging Face Spaces.

IX. DISCUSSION AND LIMITATIONS

Synthetic training targets. The LoRA model is trained on Sobel-based synthetic heightmaps rather than measured tactile ground truth from profilometers or 3D scanners. This limits physical realism: the model learns to reproduce gradient-energy patterns, not true surface micro-geometry. Collecting ground-truth (texture image, laser scan) pairs for DTD categories would be a significant but tractable data collection effort.

Single-view conditioning. The diffusion model receives a single 2D photograph of the texture. For anisotropic textures (woven, braided), the apparent surface height depends on viewing angle. Multi-view conditioning or polarization imaging [22] could resolve this ambiguity.

Resolution. LoRA fine-tuning at 512^2 produces heightmaps at that resolution, which maps to approximately 50 pixels/mm at typical fabrication scales (10mm object, 512^2 texture). For objects requiring finer than $20\mu\text{m}$ resolution (e.g., fingerprint-resolution textures), progressive upsampling or a Super-Resolution post-process [5] would be needed.

Mesh topology dependence. The bilinear heightmap sampling assumes a UV map with minimal distortion. Highly irregular or non-manifold meshes (e.g., 3D scans with holes) may produce seam artifacts. A geometry-aware sampling strategy incorporating geodesic distances would improve fidelity on complex topologies.

Future work. (1) Integrate 3DShape2VecSet [12] for end-to-end image→shape→textured-shape generation. (2) Conduct a psychophysical user study on 3D-printed objects following the TactStyle [1] protocol. (3) Extend CLIP interpolation to true image inversion using an image-conditioned decoder. (4) Investigate classifier-guided heightmap generation, conditioning the UNet on the 47-class DTD softmax distribution rather than a fixed text prompt.

X. CONCLUSION

I have presented a unified, open-source five-stage pipeline for visual and tactile texture synthesis targeted at digital fabrication. Starting from a single reference texture photograph, the system produces a textured 3D model (GLB) with genuine surface displacement geometry in under 45 seconds on a consumer GPU and under 2 seconds on a CPU via the lightweight fast path. The core technical contribution is a LoRA-adapted Stable Diffusion 1.5 heightmap generator that achieves SSIM 0.387, LPIPS 0.334, and FID 51.2 on DTD — improvements of 24%, 21%, and 25% over a gradient-based Sobel baseline. I additionally characterize three neural style transfer methods (Gatys, Johnson, AdaIN) and three texture interpolation modes (pixel, AdaIN feature, CLIP), and provide an ablation over LoRA rank showing that rank 4 is optimal for this task. The complete source code, training scripts, model checkpoints, six Jupyter notebooks, and an interactive Gradio demonstration are available at <https://github.com/omprkash/texture-model-stylization>.

ACKNOWLEDGMENT

The author thanks the creators of TactStyle, DTD, AdaIN, 3DShape2VecSet, and the diffusers library for making their work publicly available, and acknowledges Vellore Institute of Technology for providing computational resources.

REFERENCES

- [1] K. Dogan, A. Shilkrot, and P. Baudisch, “TactStyle: Generating Tactile Textures with Generative AI for Digital Fabrication,” in *Proc. ACM CHI Conf. Human Factors in Computing Systems*, 2025, doi: 10.1145/3706598.3713740.
- [2] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE/CVF CVPR*, pp. 770–778, 2016.
- [3] M. Cimpoi, S. Maji, I. Kokkinos, S. Mohamed, and A. Vedaldi, “Describing Textures in the Wild,” in *Proc. IEEE/CVF CVPR*, pp. 3606–3613, 2014.
- [4] L. A. Gatys, A. S. Ecker, and M. Bethge, “A Neural Algorithm of Artistic Style,” *arXiv preprint arXiv:1508.06576*, 2015.
- [5] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *Proc. ECCV*, pp. 694–711, 2016.
- [6] X. Huang and S. Belongie, “Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization,” in *Proc. ICCV*, pp. 1501–1510, 2017.
- [7] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-Resolution Image Synthesis with Latent Diffusion Models,” in *Proc. IEEE/CVF CVPR*, pp. 10684–10695, 2022.
- [8] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-Rank Adaptation of Large Language Models,” in *Proc. ICLR*, 2022.
- [9] J. P. Hoskins and J. Donaldson, “xatlas: Mesh Parameterization / UV Unwrapping Library,” GitHub, 2023. Available: <https://github.com/jpcy/xatlas>
- [10] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, “Learning Transferable Visual Models From Natural Language Supervision,” in *Proc. ICML*, vol. 139, pp. 8748–8763, 2021.
- [11] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfazan, and J. Zou, “Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild,” *arXiv preprint arXiv:1906.02569*, 2019.
- [12] B. Zhang, J. Tang, M. Niessner, and P. Wonka, “3DShape2VecSet: A 3D Shape Representation for Neural Fields and Generative Diffusion Models,” *ACM Trans. Graph.*, vol. 42, no. 4, 2023 (SIGGRAPH), arXiv:2301.11445.

- [13] S. Shi, A. Raza, and T. Nakano, "Language-Guided Multimodal Texture Authoring via Generative Models," *arXiv preprint arXiv:2604.06489*, 2026.
- [14] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A Large-Scale Hierarchical Image Database," in *Proc. IEEE/CVF CVPR*, pp. 248–255, 2009.
- [15] M. Dawson-Haggerty et al., "trimesh," GitHub, 2019. Available: <https://github.com/mikedh/trimesh>
- [16] K. Pugazhendhi, "3D Affordance CLIP Embeddings," GitHub, 2024. Available: <https://github.com/karanwxliaa/3d-Affordance-CLIP-Embeddings>
- [17] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020.
- [18] Y. Li, C. Fang, J. Yang, Z. Wang, X. Lu, and M.-H. Yang, "Universal Style Transfer via Feature Transforms," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017.
- [19] A. A. Efros and T. K. Leung, "Texture Synthesis by Non-Parametric Sampling," in *Proc. IEEE ICCV*, vol. 2, pp. 1033–1038, 1999.
- [20] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative Adversarial Nets," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2014.
- [21] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *Proc. ICLR*, 2014.
- [22] T. Yang et al., "Binding Touch to Everything: Learning Unified Multimodal Tactile Representations," *arXiv preprint arXiv:2401.18084*, 2024.
- [23] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis," in *Proc. ECCV*, 2020.
- [24] B. Poole, A. Jain, J. T. Barron, and B. Mildenhall, "DreamFusion: Text-to-3D Using 2D Diffusion," in *Proc. ICLR*, 2023.
- [25] D. Ulyanov, A. Vedaldi, and V. Lempitsky, "Instance Normalization: The Missing Ingredient for Fast Stylization," *arXiv preprint arXiv:1607.08022*, 2016.